Chess Next-Move Prediction with N-Gram, LSTM, and Hybrid CNN+LSTM Models

Abstract

We frame chess next-move prediction as a sequence-modelling task on human play and compare three models trained on 300 000 filtered games from the Lichess Open Database (January 2017): a trigram baseline with Katz back-off, a 2-layer LSTM (5.17 M parameters) over UCI move sequences, and a hybrid CNN+LSTM (8.0 M parameters) that adds an 8×8×18 board-tensor branch. On a held-out 15 000-game test split (1.16 M positions), the hybrid achieves top-1 accuracy 0.408 and perplexity 8.03, versus 0.371 / 10.51 for the LSTM and 0.252 / 156.7 for the baseline. The most informative finding is the phase-by-phase shape: adding board awareness gives essentially zero gain in the opening but improves perplexity by 47 % in the endgame — board state matters most precisely where pure-sequence context becomes least informative. All three models support legal-move masking at inference, so they cannot recommend illegal moves.

1. Problem definition

Given a game's history, predict the next move played: we model `P(move_i | move_0, …, move_{i-1})` at every position (and, for the hybrid, additionally on the board state). This is a *behaviour-prediction* problem — we predict what humans typically play, not what is theoretically optimal. We tokenise every move using **UCI** notation: fixed-width strings such as `e2e4`, `g8f6`, `e7e8q`. UCI is unambiguous and gives a closed vocabulary covering every legal move.

Throughout we use **plies** (single half-moves; one full move = two plies). We evaluate every model with three metrics: **top-1/3/5 accuracy** (legal-move-masked), **perplexity** (`exp(mean cross-entropy)`, lower is better), and a **per-phase breakdown** — opening (plies 0–19), early middlegame (20–39), late middlegame (40–79), endgame (80+). The motivating use case is automated chess teaching: tools that play optimally are abundant; tools that predict realistic human moves at a target playing strength have far fewer instances (Maia Chess [1] is the closest prior art).

2. Data preparation

Source and filtering. Lichess Open Database, 2017-01 (≈ 1.9 GB compressed, ≈ 5 M raw games). We filter to: TimeControl ≥ 480 s (Rapid + Classical only), both Elo ≥ 1500, `Termination == "Normal"`, 40 ≤ plies ≤ 240. After filtering we keep the first 300 000 games. A custom python-chess visitor (`_HeaderFilterVisitor`) returns `SKIP` the moment the header fails a filter, before the body is decoded; since ~80 % of Lichess games are bullet/blitz that fail TimeControl, this reduced slicing time from ~60 min to ~15 min.

Tokenisation. The vocabulary contains four reserved tokens (`<PAD>` = 0, `<START>` = 1, `<END>` = 2, `<UNK>` = 3) followed by the sorted UCI moves observed in the train split, built deterministically with `seed = 42`. For our 255 000-game train split this yields 1 940 tokens (theoretical maximum ≈ 1 972). `<PAD>` is batching filler, `<START>` and `<END>` mark game boundaries, and `<UNK>` is a fallback for unseen strings (almost never used). All four are excluded from inference top-K.

Board encoding. 8×8×18 float32 tensor: 12 piece planes (P/N/B/R/Q/K, white then black) + 1 turn plane + 4 castling-rights planes + 1 en-passant target plane. We deliberately exclude tactical channels (attack maps, pins, hanging pieces) so any tactical understanding the model exhibits is genuinely learned from raw positions rather than handed in.

Splits. 85 / 10 / 5 train / val / test, partitioned *by game* (never by position) with `random.Random(seed=42).shuffle`. Sizes: 255 000 / 30 000 / 15 000 games — yielding ≈ 19.78 M / 2.33 M / 1.16 M individual position-prediction samples.

The LSTM trains on whole games (one game per sample, variable length); the hybrid trains on individual positions (one per ply, each carrying a board snapshot, the move history up to that point, and the move actually played next). The hybrid needs a board picture per sample, hence the per-position view.

3. Models

N-gram baseline (`TrigramKatz`, pure Python). Trigram counts over the training games with leading `<START><START>` and trailing `<END>` per game. Inference: `P(c | a, b) = (count − d) / T` with absolute discount `d = 0.5`; back-off to bigram, then to a Laplace-smoothed unigram so probabilities are never zero. Fitted model: 1 870 unigrams, 175 144 bigram contexts, several million trigram contexts (37 MB gzipped).

LSTM (`MoveLSTM`, 5.17 M params). Embedding (256-d) ⟶ 2-layer LSTM (hidden 512, dropout 0.2) ⟶ linear projection to 1 940 logits. Standard left-to-right next-token prediction at training; at inference, the hidden state after the last played move feeds the logit head.

Hybrid CNN + LSTM (`MoveBoardHybrid`, 7.99 M params). Same LSTM branch over move history, plus a 3-layer CNN over the 8×8×18 board (channels 18 ⟶ 64 ⟶ 128 ⟶ 128, ReLU, same-padded 3×3 throughout). Flatten + linear projection to a 256-d board feature, concatenated with the LSTM's 512-d summary, then a final linear layer to next-move logits. Intuition: the LSTM branch captures *what kind of game this is* (opening structure, plan progression); the CNN branch captures *what the position looks like right now* (piece relationships, endgame patterns). The shared head learns to weight the two signals.

Legal-move masking. All three share a `predict_topk(history, k, legal_token_ids)` interface. Models can assign internal probability mass to illegal moves, but special tokens and illegal moves are filtered before output — the worst possible failure is a strategically poor *legal* move, never an illegal one.

4. Experimental setup

Both neural models share AdamW, learning rate 1×10^{-3}, weight decay 0, gradient clip 1.0, ReduceLROnPlateau (factor 0.5, patience 1) on val loss, and CrossEntropy loss with `<PAD>` ignored. The LSTM trains 8 epochs at batch 64 (whole games per step); the hybrid trains 4 epochs at batch 256 (individual positions per step). The hybrid's 4 epochs is comparable to the LSTM's ~320 epochs in samples seen, because each game contributes one sample to the LSTM but ~80 to the hybrid (one per ply). Best-by-val-loss checkpoints are kept.

A uniform `Predictor` interface (`src/training/evaluate.py`) ensures all three models go through the same aggregation code: top-K accuracy, perplexity, phase-bucketed metrics. All randomness is seeded from `seed = 42` (splits, vocab order, per-epoch shuffling). The codebase has 82 passing unit tests covering the pipeline, models, and harness.

5. Results

Three-way comparison on the held-out test split (1 162 103 position-prediction pairs, same vocabulary across all models):

Metric	n-gram	LSTM	Hybrid	LSTM vs n-gram	Hybrid vs LSTM
Top-1	0.252	0.371	**0.408**	+11.9 pp	+3.7 pp
Top-3	0.458	0.630	**0.675**	+17.3 pp	+4.5 pp
Top-5	0.563	0.744	**0.784**	+18.1 pp	+4.0 pp
Perplexity	156.7	10.51	**8.03**	14.9× lower	24 % lower

Per-phase breakdown (top-1 accuracy / perplexity):

Phase (plies)	n positions	n-gram	LSTM	Hybrid	n-gram pp	LSTM pp	Hybrid pp
Opening (0–19)	300 000	0.366	**0.474**	0.473	18.0	5.37	5.34
Early-mid (20–39)	300 000	0.213	0.369	**0.393**	163	9.81	8.56
Late-mid (40–79)	401 899	0.187	0.303	**0.361**	540	16.4	10.8
Endgame (80+)	160 204	0.275	0.352	**0.431**	373	13.8	7.35

Generalisation. LSTM val/test perplexity 10.62 / 10.51; hybrid 8.04 / 8.03. No overfitting signal. Both validation curves were still falling at their cut-off epochs.

6. Performance analysis

The two cleanest findings emerge from the per-phase breakdown.

The LSTM closes the n-gram's middlegame gap. The n-gram peaks in the opening (top-1 0.366) — exactly where memorisation matters: with millions of trigram contexts seen across 255 000 games, common opening continuations are well-covered. Where it fails is the middlegame:

top-1 collapses to 0.187 in late-middlegame because moves 21–40 diverge into a long tail of unique configurations a three-token window cannot encode. The LSTM closes that gap — its biggest absolute gains over the baseline are in early-middlegame (+0.156 top-1) and late-middlegame (+0.116 top-1). Full-game history compressed into the hidden state encodes opening choice, plan execution, and trade dynamics in a way the trigram window cannot.

The hybrid's gain over the LSTM grows monotonically through the game. This is the more interesting finding because it is *predicted by first principles*: board awareness should add the most value precisely where pure-sequence context becomes least informative.

Phase	Hybrid top-1 Δ vs LSTM	Hybrid pp improvement over LSTM
Opening	−0.001 (tied)	tied (5.34 vs 5.37)
Early-middlegame	+0.024	13 % (8.56 vs 9.81)
Late-middlegame	+0.058	34 % (10.8 vs 16.4)
Endgame	+0.079	**47 %** (7.35 vs 13.8)

In the opening, the LSTM and hybrid are tied. Adding the board tensor and CNN contributes essentially nothing because openings are *defined* by their move sequences — any board state reachable from a given opening is implied by it. By endgame the picture is opposite: move history matters less (most moves were trades that have already resolved), and the surviving-piece configuration becomes the dominant signal. Hybrid endgame top-1 0.431 vs LSTM 0.352 (+7.9 pp), perplexity 7.35 vs 13.8 (47 % better).

This phase-shape *falsifies* the alternative hypothesis ("just adding more parameters is what helped"). If raw parameter count were doing the work, the gain should be roughly uniform across phases. Monotonic phase-shape is direct evidence that the *board signal*, not added capacity, drives the late-game improvement.

7. Limitations and future work

No multi-move planning. None of the three models reliably finds even simple checkmates. K+Q vs K is mated by any 1500+ rated human in seconds via short-horizon search and pattern recognition; our models predict one move at a time from learned single-step distributions, with no search and no multi-move plan representation. Search-based methods (MCTS, AlphaZero-style) would close this gap and are a natural extension, but were out of scope for a project concerned with prediction of human play rather than optimal play.

Distribution sensitivity. The training corpus is filtered to coherent, skilled play (Rapid/Classical, both Elo ≥ 1500, normal termination, single Lichess month). Predictions degrade outside that distribution: deliberately constructed odd positions, sequences with multiple intentional blunders, or positions imported from unrelated game formats. Cross-month and cross-Elo-band generalisation were not investigated.

Choices we did not pursue. The project specification excluded (and we did not implement): Elo-conditioned models in the style of Maia [1], hand-engineered tactical input channels, board-

mirroring augmentation, Stockfish-based "best-move" comparison, calibration analysis (ECE), and FAISS retrieval. The most promising open continuation, motivated by Karvonen's analysis of LLMs on chess [2], is grounded natural-language explanation of moves: combining a Maia-style human-prediction layer with a fine-tuned language model that generates rationale text, validated against an engine to suppress hallucinated tactical claims.

8. Conclusion

Three increasingly informed models trained on the same data and evaluated on the same held-out test split show monotone improvement at each step. The n-gram establishes the coverage of pure local pattern matching (top-1 0.252, perplexity 156.7) — strong in the opening, weak in the middlegame. The LSTM adds full-game history and closes the middlegame gap (top-1 0.371, perplexity 10.51). The hybrid CNN+LSTM adds board awareness and consistently improves on the LSTM (top-1 0.408, perplexity 8.03). The hybrid's improvement is phase-shaped: tied with the LSTM in the opening, growing through the middlegame, largest in the endgame (+7.9 pp top-1, 47 % better perplexity). This phase shape is direct evidence that the board signal earns its parameters precisely where sequence context becomes least informative. The codebase is fully unit-tested (82 tests), seeded, and reproducible from the public Lichess monthly. None of the three models reliably finds basic checkmates — predicting *what humans tend to play* is a different problem from *playing optimally*.

References

[1] McIlroy-Young, R., Sen, S., Kleinberg, J., & Anderson, A. (2020). *Aligning Superhuman AI with Human Behavior: Chess as a Model System.* KDD '20.

[2] Karvonen, A. (2024). *Examining GPT-3.5-turbo-instruct's Chess Skill.* https://www.adamkarvonen.com/machine_learning/2024/01/03/chess-world-models.html

[3] Lichess Open Database. https://database.lichess.org

Code: https://github.com/kornel9/chess-move-prediction